

MODULE 1

Setup & Installation Guide

User Identity, Profile & Portfolio Management

National Freelance & Skill Verification Platform

FAST NUCES • Semester 6 • SPM Project • 2026

Stack: PostgreSQL • Express.js • React (Vite) • Node.js

1. Project Overview

This README will help you clone, install, configure, and run Module 1 of the National Freelance & Skill Verification Platform on your local Windows machine. Follow every step in order. Do not skip any step.

What is Module 1?

Module 1 handles User Identity, Profile & Portfolio Management. It is the authentication and profile backbone of the entire platform. Every other module depends on it for user identity, JWT tokens, and profile data.

Component	Technology	Port	Purpose
Database	PostgreSQL 15+	5432	Stores all user, profile, skills, reviews data
Backend	Node.js + Express	5001	REST API server, JWT auth, bcrypt
Frontend	React + Vite	5173	User interface — login, register, dashboard

2. Prerequisites

Install all three tools below before proceeding. All are free and open-source.

2.1 Node.js

Download and install Node.js v18 or higher from the official website.

```
https://nodejs.org/en/download
```

After installing, verify it works by opening PowerShell and running:

```
node --version  
npm --version
```

Expected Output

node --version should show v18.x.x or higher npm --version should show 9.x.x or higher

2.2 PostgreSQL

Download PostgreSQL version 15 or higher for Windows from:

```
https://www.postgresql.org/download/windows/
```

During installation, set the following:

- Password for the postgres superuser — remember this, you will need it
- Port — keep it as 5432 (default)

- pgAdmin 4 — make sure this is checked in the installer

After installing, verify PostgreSQL is running by opening PowerShell:

```
psql -U postgres -c "SELECT version();" 
```

Expected Output

PostgreSQL 16.x on x86_64-pc-windows should appear in the response.

3. Getting the Project

3.1 Verify Folder Structure

After cloning or extracting, confirm the folder looks like this:

```
module1-identity/
  backend/
    src/
      controllers/
      routes/
      middleware/
      models/
      services/
      config/
        db.js
      utils/
      server.js
      .env.example
      package.json
  database/
    schema.sql
    triggers.sql
  frontend/
    src/
      pages/
      components/
      context/
      services/
    package.json
    .env.example
```

4. Database Setup

4.1 Create the Database

Open PowerShell and run the following command. It will ask for your PostgreSQL password:

```
psql -U postgres -c "CREATE DATABASE SPM_db;"
```

If you get an error

Make sure PostgreSQL is running. Open Windows Services (press Win + R, type services.msc) and look for postgresql-x64-15. Right click it and click Start.

4.2 Run the Schema

Navigate to the database folder inside the project. Replace YourName with your actual Windows username:

```
cd "C:\Users\YourName\Desktop\module1-identity\database"
psql -U postgres -d SPM_db -f schema.sql
```

You will be asked for your PostgreSQL password. After entering it, you should see output like this:

```
CREATE TYPE
CREATE TYPE
CREATE TABLE
CREATE TABLE
CREATE INDEX
... (many more lines)
```

4.3 Run the Triggers

```
psql -U postgres -d SPM_db -f triggers.sql
```

Expected output:

```
CREATE FUNCTION
CREATE TRIGGER
```

4.4 Verify Tables in pgAdmin

- Open pgAdmin 4 from the Start Menu
- Connect to your local server with your postgres password
- Expand: Servers → PostgreSQL → Databases → SPM_db → Schemas → public → Tables
- Right-click Tables and click Refresh
- You should see tables like users, profiles, skills, reviews, portfolio_projects, etc.

Tables not showing?

Right-click at each level (Databases, SPM_db, Schemas, public, Tables) and click Refresh at each level. pgAdmin does not auto-refresh.

5. Backend Setup

5.1 Navigate to Backend Folder

```
cd "C:\Users\YourName\Desktop\module1-identity\backend"
```

5.2 Install Dependencies

This installs all the Node.js packages the backend needs:

```
npm install
```

This installs the following packages automatically:

Package	Purpose
express	Web server framework — handles all HTTP requests
pg	PostgreSQL client — connects Node.js to the database
bcryptjs	Password hashing — stores passwords securely
jsonwebtoken	JWT creation and verification for authentication
dotenv	Loads environment variables from the .env file
cors	Allows the React frontend to call the backend API
multer	Handles file uploads like profile pictures
express-validator	Validates incoming request data
nodemon	Dev tool — auto-restarts server on file save

5.3 Create the .env File

The .env file contains your secret configuration. Never share this file or commit it to Git.

Create a new file called .env inside the backend folder. You can do this in VS Code or with Notepad. Paste the following content and fill in your own values:

```
PORT=5001
DB_HOST=localhost
DB_PORT=5432
DB_NAME=SPM_db
DB_USER=postgres
DB_PASSWORD=your_actual_postgres_password_here
JWT_SECRET=module1_nexus_super_secret_key_2026
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:5173
```

Important

Replace your `your_actual_postgres_password_here` with the password you set when you installed PostgreSQL. The `JWT_SECRET` can be any long random string — make it unique.

5.4 Start the Backend Server

```
npm run dev
```

You should see the following output in your terminal:

```
[nodemon] starting `node server.js`  
Server running on port 5001  
PostgreSQL connected
```

Server crashed?

Most common reasons: 1. Wrong DB_PASSWORD in .env 2. PostgreSQL service is not running — check Windows Services 3. A route file is missing from src/routes/ — check all 6 route files exist 4. Port 5001 is already in use — change PORT in .env to 5002

5.5 Test the Backend

Open your browser and go to:

```
http://localhost:5001
```

You should see this JSON response in the browser:

```
{ "message": "Module 1 API running" }
```

6. Frontend Setup

6.1 Open a New Terminal

Keep the backend terminal running. Open a second PowerShell window for the frontend.

6.2 Navigate to Frontend Folder

```
cd "C:\Users\YourName\Desktop\module1-identity\frontend"
```

6.3 Install Dependencies

```
npm install
```

This installs these packages:

Package	Purpose
react	Core React library
react-dom	Renders React components in the browser
react-router-dom	Client-side routing between pages
axios	Makes HTTP requests to the backend API

Package	Purpose
react-hook-form	Manages form state and validation
yup	Schema-based form validation rules
vite	Fast development build tool (replaces CRA)

6.4 Create the Frontend .env File

Create a file called .env inside the frontend folder with this content:

```
VITE_API_URL=http://localhost:5001/api
```

This tells React where your backend API lives. If you changed the backend port, update this accordingly.

6.5 Start the Frontend

```
npm run dev
```

You should see:

```
VITE v5.x.x  ready in 300ms
Local:      http://localhost:5173/
Network:    http://192.168.x.x:5173/
```

Open your browser and go to <http://localhost:5173> — you should see the Nexus Professional login page with the animated dark background.

7. Running the Full Stack

You need three things running simultaneously for the project to work fully:

What	How to Start	Port	Status Check
PostgreSQL	Runs as Windows service	5432	Open pgAdmin — should connect
Backend API	npm run dev in backend/	5001	http://localhost:5001 shows JSON
React Frontend	npm run dev in frontend/	5173	http://localhost:5173 shows login page

Quick Start Summary

Terminal 1 (Backend): `cd module1-identity/backend npm run dev` Terminal 2 (Frontend): `cd module1-identity/frontend npm run dev` PostgreSQL runs automatically as a Windows service — no terminal needed.

8.3 Test the API Directly (Optional)

You can test backend endpoints directly using Thunder Client in VS Code or Postman.

Endpoint	Method	Body Required	Auth
http://localhost:5001/api/auth/register	POST	first_name, last_name, email, password, role	None
http://localhost:5001/api/auth/login	POST	email, password	None
http://localhost:5001/api/auth/me	GET	None	Bearer token
http://localhost:5001/api/auth/logout	POST	None	Bearer token
http://localhost:5001/api/auth/change-password	POST	oldPassword, newPassword	Bearer token